

Tema 04. Diseño de sistemas de Aprendizaje Automático

Autor: Ismael Sagredo Olivenza

4.1 Proceso de desarrollo del ML

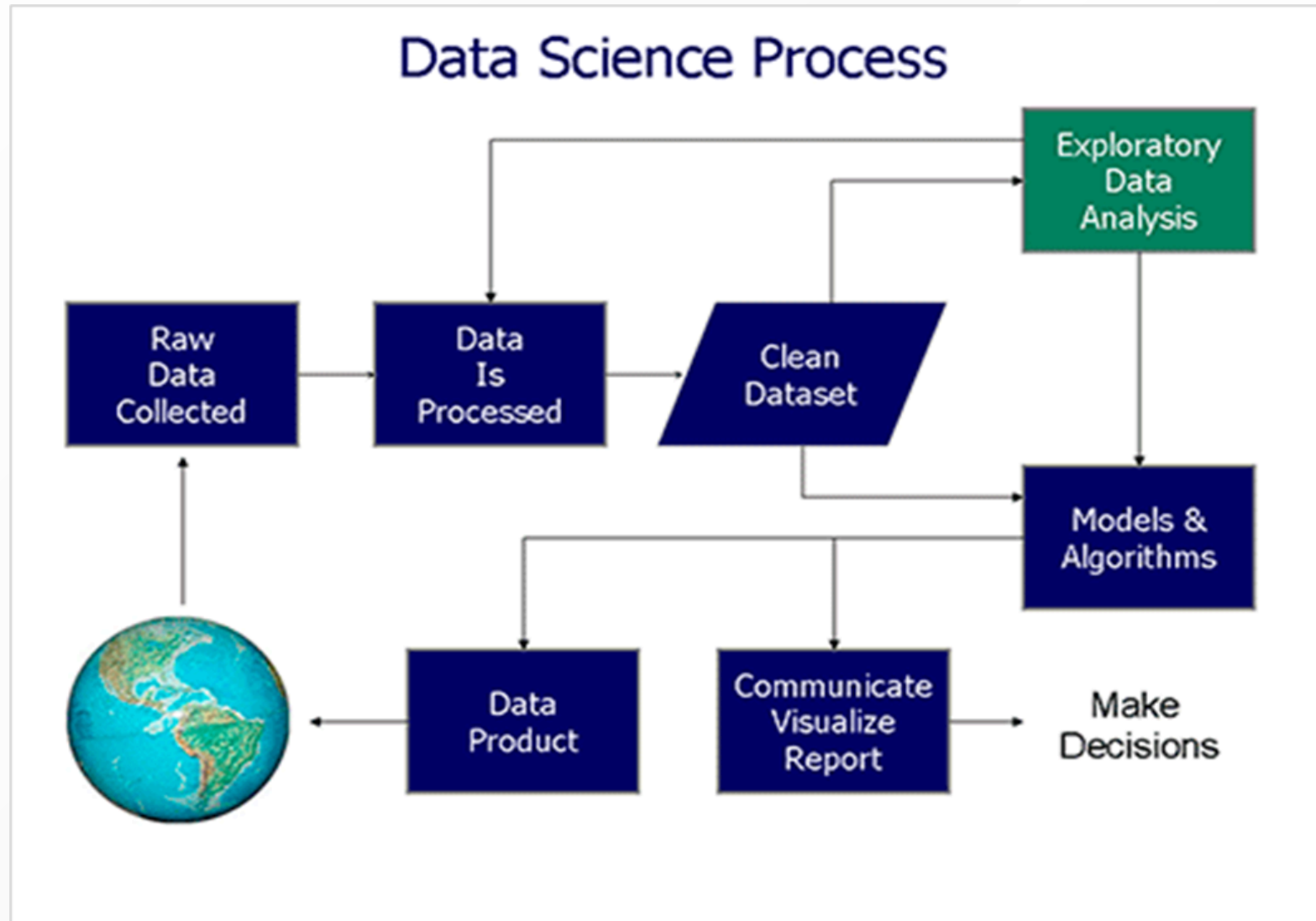
En proyectos reales:

Gran parte del tiempo de desarrollo se dedica a obtener, pulir y limpiar datos.

Los algoritmos de ML suelen tener que volver a entrenarse para mantener su relevancia, ya que el entorno cambia a menudo. Y durante el entrenamiento hay que hacer muchos ajustes finos.

Así, tenemos un proceso iterativo de desarrollo de proyectos basados en aprendizaje automático.

4.1.1 Data Science Process



Este proceso iterativo se refleja en el **Proceso de Ciencia de Datos**. En muchas ocasiones, la modificación del mundo por parte de la propia IA hace que el entorno cambie, por lo que debemos repetir el proceso de nuevo.

Si nos centramos únicamente en el proceso iterativo desde el punto de vista del Machine Learning, podemos simplificar el proceso a tres fases:

- Elegir la arquitectura: incluyendo datos, modelos, etc.
- Entrenar el modelo: Entrenamiento del propio modelo, selección de hiperparámetros, etc.
- Diagnóstico: sesgo, varianza, error, etc.

4.2 Limpiar y codificar los datos

Limpiar el conjunto de datos de errores y valores nulos. Estrategias:

- Eliminar observaciones: Si hay pocos datos no es recomendable.
- Imputar al campo o característica un valor que tenga sentido según los datos disponibles. Por ejemplo el valor medio / moda

```
from sklearn.impute import SimpleImputer
```

- Imputar el valor medio de las entradas más similares al dato que estamos inputando (KNNImputer)

```
from sklearn.impute import KNNImputer
```

4.2.1 One-hot encoding

Al trabajar con variables categóricas, una de las cuestiones más importantes es transformar nuestras variables categóricas en variables numéricas.

One-Hot Encoding transforma una característica categórica en tantas variables como valores posibles tenga la categoría y asigna a cada una de ellas 0 ó 1 en función del valor.

```
from sklearn.preprocessing import OneHotEncoder
oneHotEncoder = OneHotEncoder(drop='first')
oneHotEncoderFit = oneHotEncoder.fit(provincias.reshape(-1,1))
oneHotEncoderFit.transform(provincias.reshape(-1,1)).toarray()
```

4.3 Evaluación del modelo

- **Mean Square Error** entre el pronóstico y el real.

$$MSE(Y, Y') = \frac{1}{N} \cdot \sum_{i=1}^N (y_i - y'_i)^2$$

- **Confussion Matrix** (Ejemplo para clase binaria)

	C observed T	C observed F
C Predicted T	True Positive (TP)	False Positive (FP)
C Predicted F	False Negative (FN)	True Negative (TN)

- **Accuracy** o tasa de exactitud: Aciertos entre el total.

$$Accuracy = \frac{TP + TN}{TP + N + FP + FN} = \frac{V}{N}$$

La exactitud puede presentar problemas en los casos en que las clases estén desequilibradas.

- True Positive Rate / **Sensitivity (Recall)**. Verdaderos positivos entre todos los positivos observados (de todos los positivos reales (TP + FN), cual es el ratio de positivos detectados.)

$$Recall = \frac{TP}{TP + FN}$$

- Positive Predictive Value (**Precision**). Verdaderos positivos entre todos los positivos predecidos. (De todo lo que nuestro clasificador clasifica como positivo (TP + FP), cual es el ratio de clasificaciones correctas)

$$Precision = \frac{TP}{TP + FP}$$

		Actual Outcome	
		Positive	Negative
Predicted Outcome	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Positive Predicted Value = $TP / (TP + FP)$

		Actual Outcome	
		Positive	Negative
Predicted Outcome	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Sensitivity = $TP / (TP + FN)$

Equilibrio entre Precision y Recall

Las métricas de precision y recall están relacionadas de manera que si entrenas tu clasificador para aumentar la precisión, disminuirá el recall y viceversa.

Una métrica que mide el equilibrio de ambas es la siguiente:

- **F1 Score:** entre 0 y 1. 1 sería un clasificador perfecto.

$$F1Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$$

Ejemplo

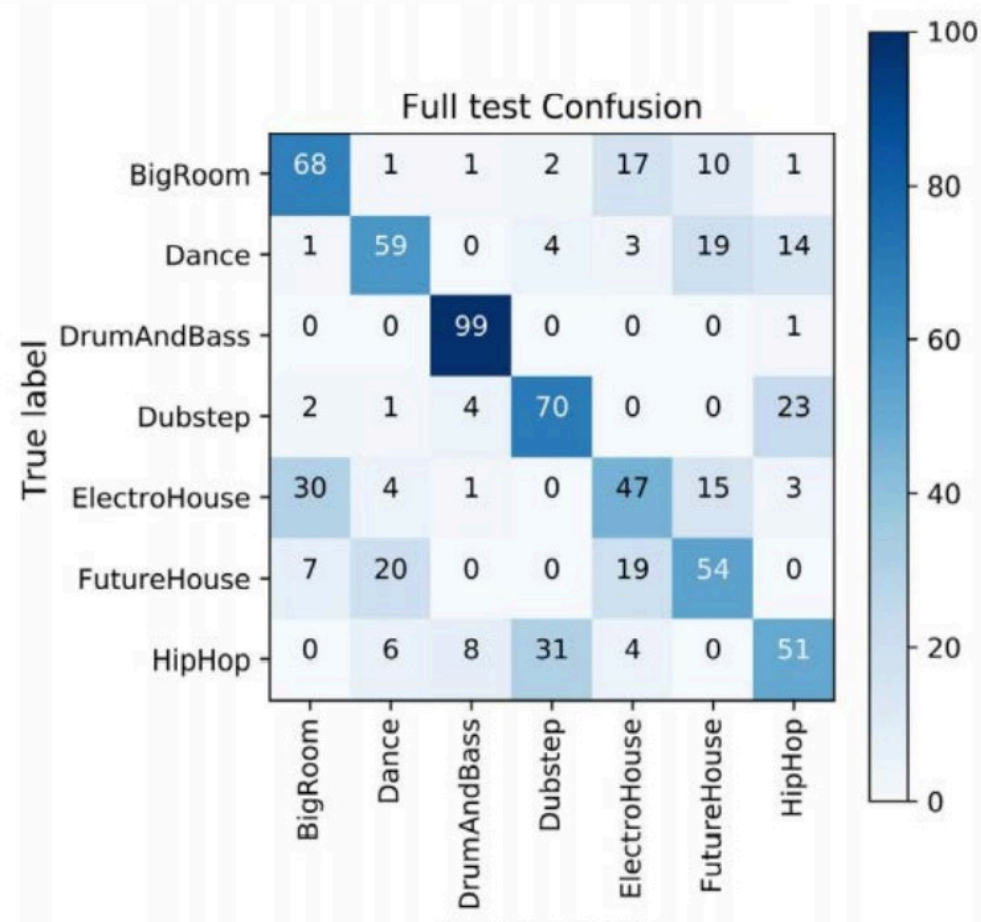
Confussion Matrix: Calcula precision, recall, F1Score.

	C observed T	C observed F
C Predicted T	176 (TP)	107 (FP)
C Predicted F	316 (FN)	993 (TN)

- Precision: $precision = \frac{TP}{TP+FP} = \frac{176}{176+107} = 0,6219 = 62,19\%$
- Recall: $recall = \frac{TP}{TP+FN} = \frac{176}{176+316} = 0,3577 = 35,77\%$
- F1Score: $F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall} = 2 \cdot \frac{0.62 \cdot 0.35}{0.62+0.35} = 0,4488$

Ejercicio

Interpreta la siguiente matriz de confusión. Clasificar canciones según el subgénero de música electrónica al que pertenecen



4.3.1 Testing

Dividir el conjunto de datos en datos de entrenamiento y datos de prueba.

La distribución de los datos entre prueba y entrenamiento debe ser correcta y proporcional al número de clases. Una distribución sesgada (Biases) puede causar problemas.

SKLearn nos puede ayudar en esta tarea:

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(data, y, train_size = 0.8, random_state = 1234)
```

Durante la fase de entrenamiento, no sólo debemos fijarnos en el error de entrenamiento, sino también en el **error de validación**. El error de validación sería similar al **error de prueba**, pero se ejecuta durante el entrenamiento para ver si el modelo está perdiendo capacidad de generalización.

A priori podría parecer que la validación y la prueba son lo mismo, pero estos dos conjuntos se suelen diferenciar porque cuando se prueban diferentes ajustes del modelo (hiperparámetros) existe **aún un riesgo de sobreajuste** en el **conjunto de prueba** porque los parámetros se pueden **ajustar hasta que el estimador funcione de forma óptima**. De este modo, el conocimiento sobre el conjunto de pruebas puede "filtrarse" en el modelo y las métricas de evaluación ya no informan sobre el rendimiento de la capacidad de generalización.

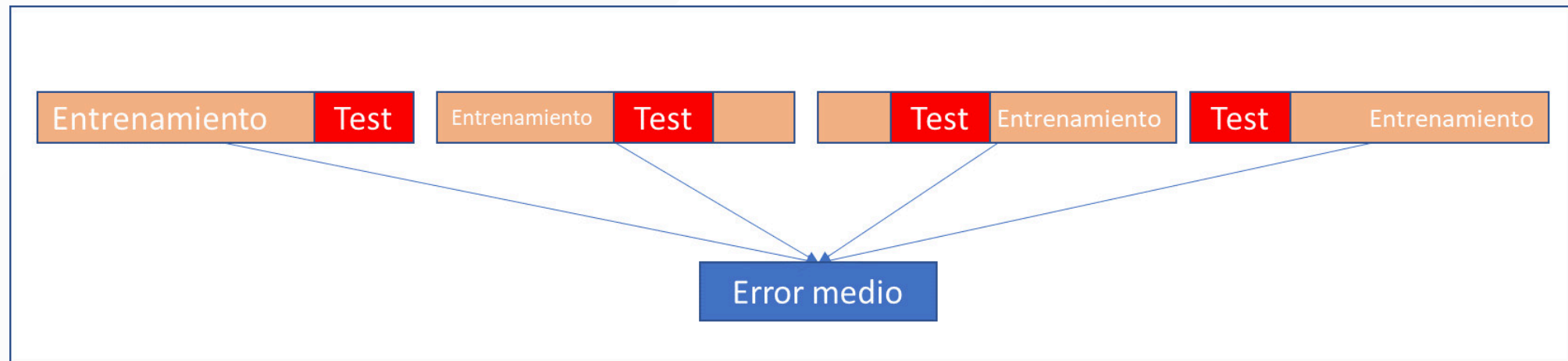
Por lo tanto, tendremos los siguientes conjuntos:

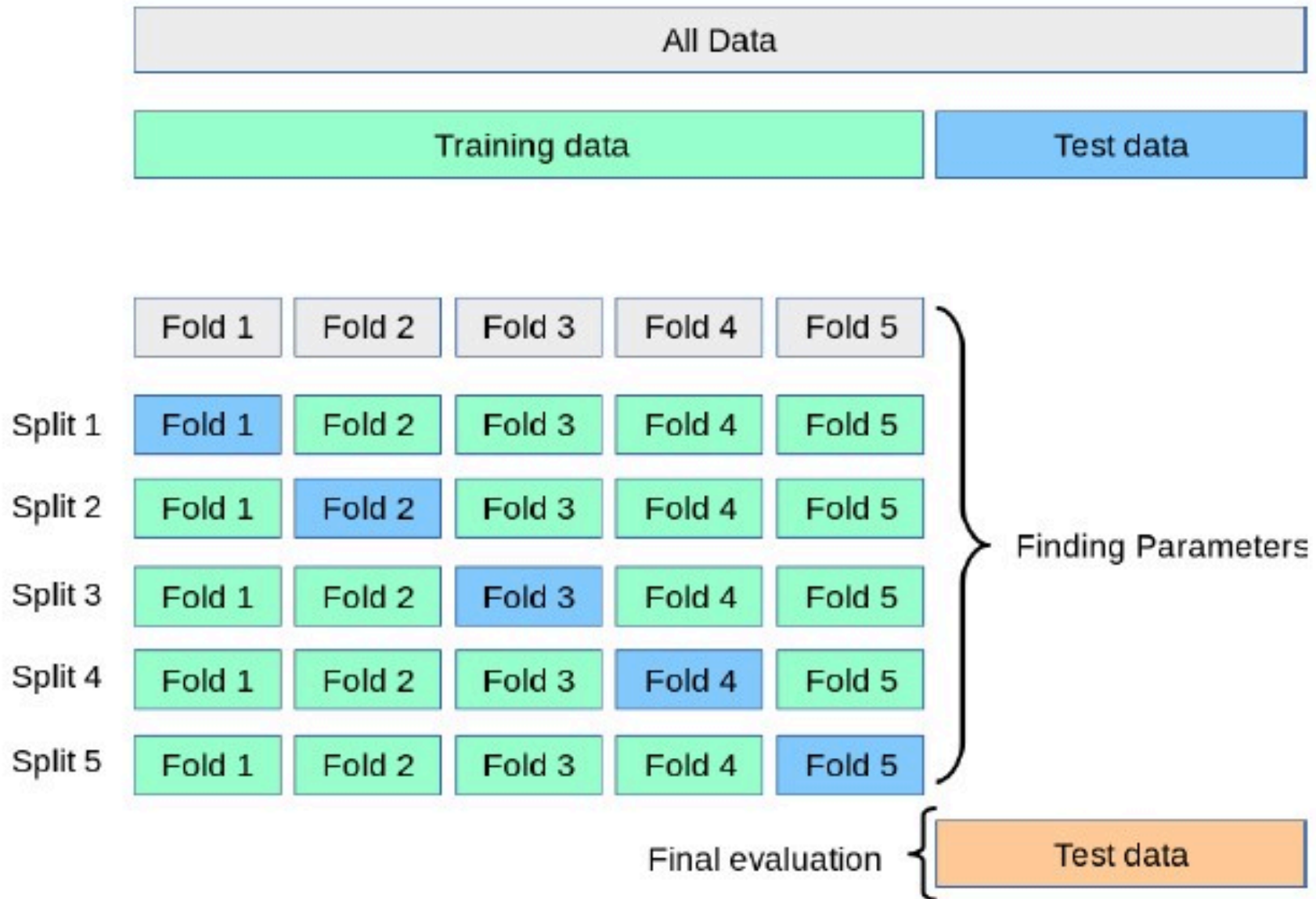
- **Error de entrenamiento:** Error de datos de entrenamiento.
- **Error de validación** (validation): Error de validación durante el entrenamiento para mantener la capacidad de generalización.
- **Error de pruebas** (test): Error con datos completamente excluidos del entrenamiento que tiene lugar después del entrenamiento.

Sin embargo, al dividir los datos disponibles en tres conjuntos, reducimos drásticamente el número de muestras que pueden utilizarse para el aprendizaje del modelo, y los resultados pueden depender de una elección aleatoria concreta para el par de conjuntos (entrenamiento, validación). Una solución es la **validación cruzada** o **cross-validation**

K-Fold Cross Validation

Cross-validation nos permite dividir los datos en k subgrupos y utilizar para el entrenamiento $k-1$ de ellos y para la validación durante el entrenamiento el restante. El error final es el error medio. Esto nos permite aprovechar los datos de entrenamiento y evitar posibles sesgos en la elección del conjunto de validación.





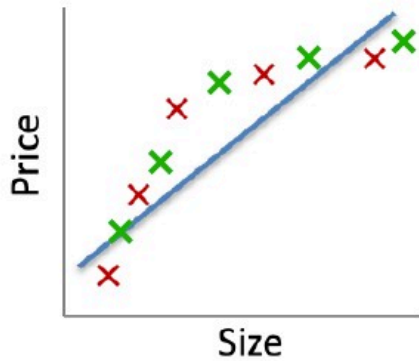
```
from sklearn.model_selection import cross_val_score
clf = Stimator(For example MLPClassifier)
scores = cross_val_score(clf, X, y, cv=5)
scores
print("%0.2f accuracy with a standard deviation of %0.2f" % (scores.mean(), scores.std()))
```

Por defecto, la puntuación calculada en cada iteración de CV es el método de puntuación del estimador. Es posible cambiar esto utilizando el parámetro de puntuación:

```
scores = cross_val_score(clf, X, y, cv=5, scoring='f1_macro')
```

- Leave One Out (LOO)
- Leave P Out (LPO), Leave P Out (LPO)
- ShuffleSplit...

4.4 Diagnostic Bias and Variance



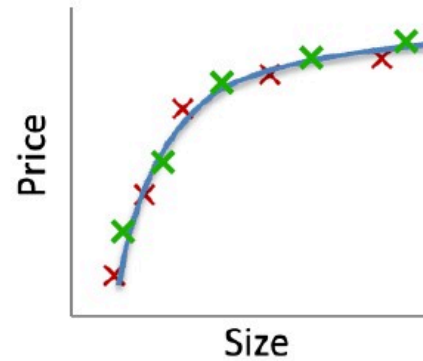
$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + b$$

High bias
(underfit)

$$d = 1$$

J_{train} is high

J_{cv} is high



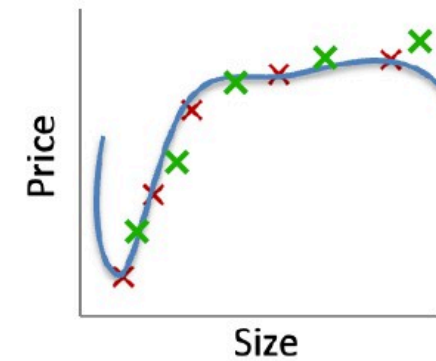
$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x^2 + b$$

"Just right"

$$d = 2$$

J_{train} is low

J_{cv} is low



$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x^2 + w_3x^3 + w_4x^4 + b$$

High variance
(overfit)

$$d = 4$$

J_{train} is low

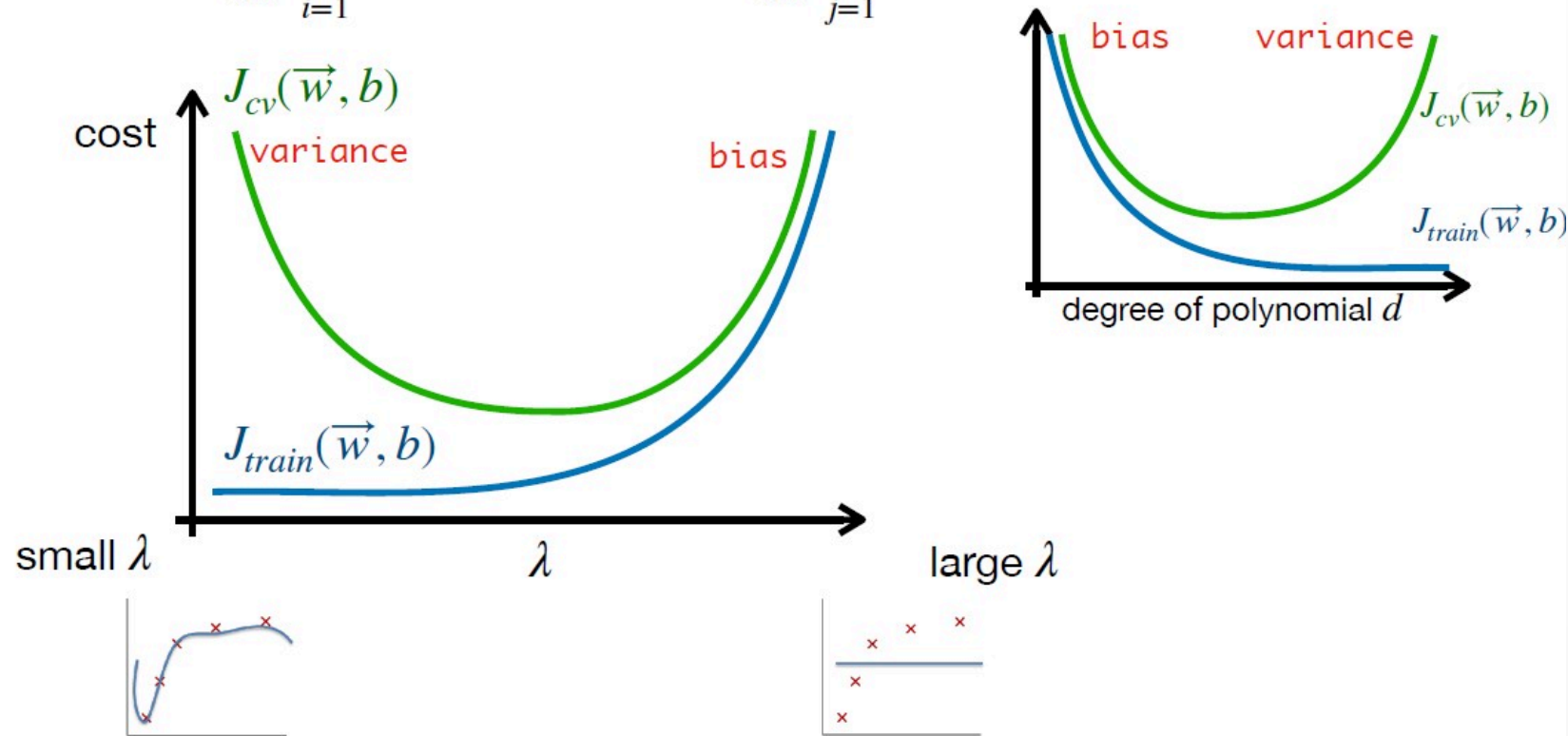
J_{cv} is high

4.4.1 Bias / varianza en función del parámetro de regularización

Cuanto mayor sea el parámetro de regularización, la diferencia entre bias y varianza se reducirá. Lo importante es encontrar el punto correcto donde la varianza sea la mas baja posible, ya que no nos interesa que crezca. Ese es el punto correcto del parámetro λ

Bias/variance as a function of regularization parameter λ

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

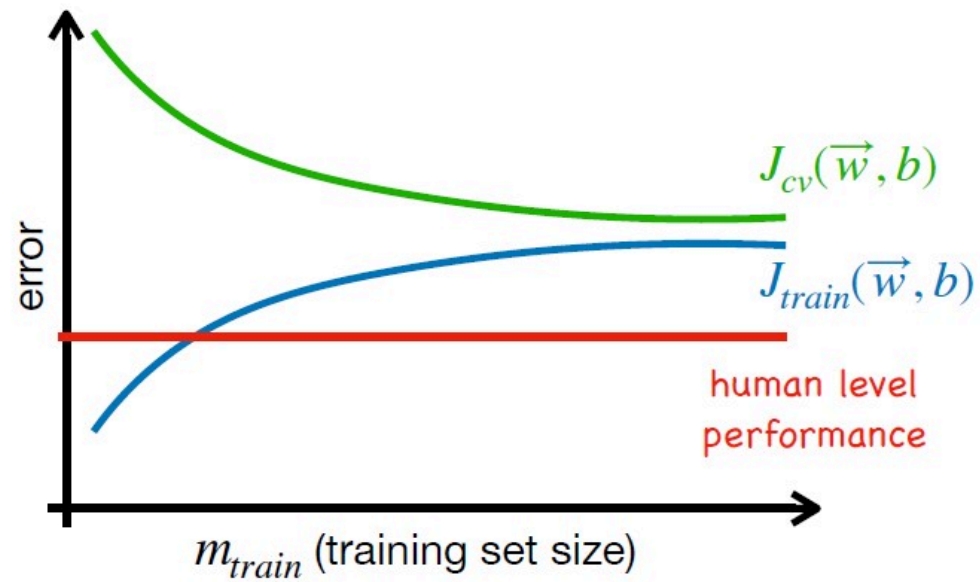


One of the challenges with building machine learning systems is that there's so many things you could try, so many things you could change (hyperparameters)

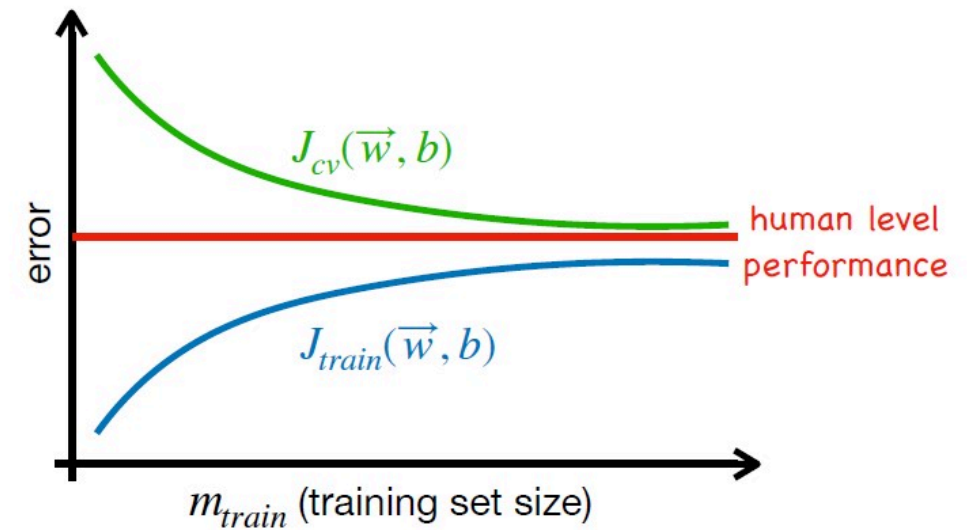
4.4.2 Podemos usar una baseline para medir el rendimiento.

Usando una baseline o medida de lo que consideramos suficientemente bueno o lo que pretendemos superar (por ejemplo el rendimiento humano en una tarea), podemos estimar si el modelo necesita más complejidad o mas ejemplos. Si tanto el error de entrenamiento como el de validación están por encima del error humano, necesitaremos un modelo más complejo. Si la varianza es alta, necesitamos más datos.

Usando **accuracy** sería similar pero a la inversa, si tengo mayor accuracy que el humano mejor.

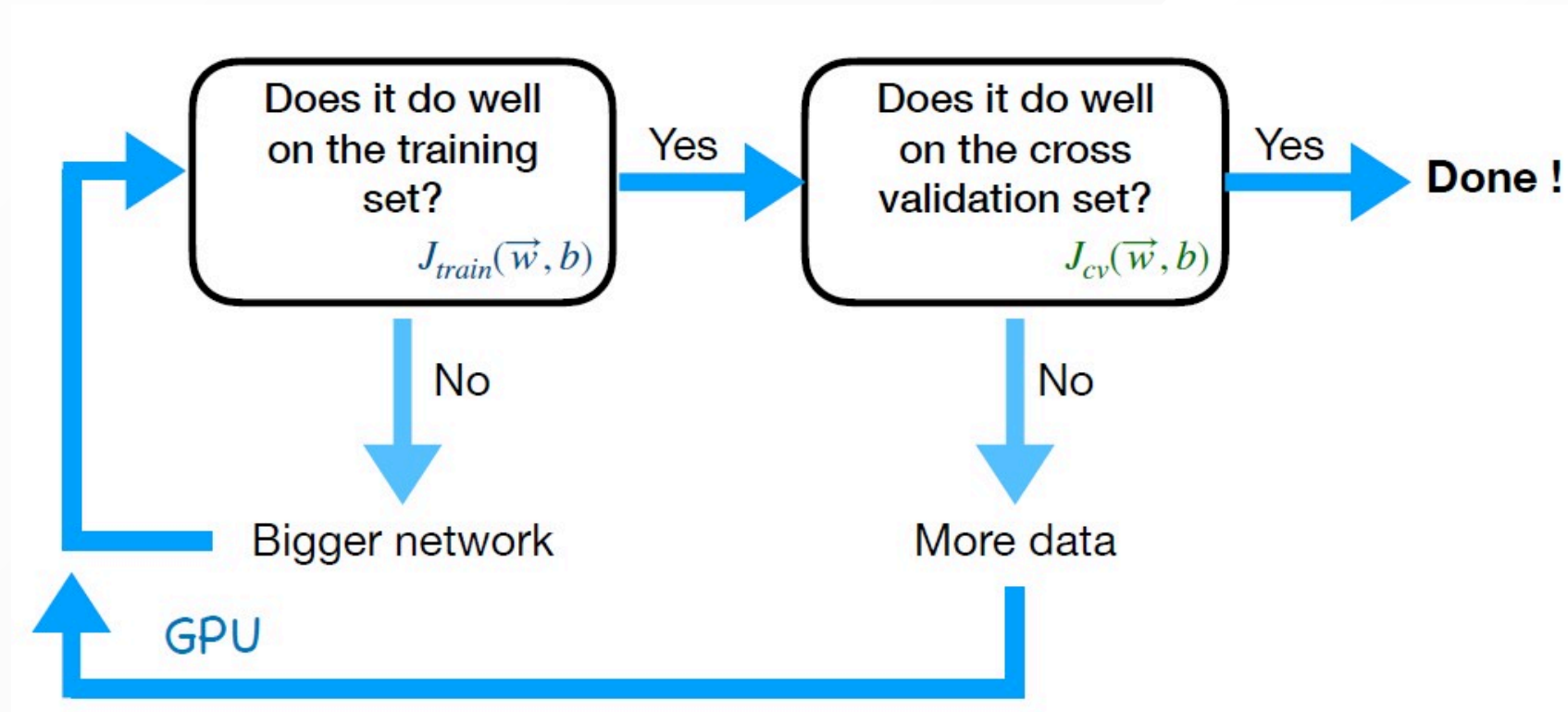


if a learning algorithm suffers from high bias, getting more training data will not (by itself) help much



if a learning algorithm suffers from high variance, getting more training data is likely to help

4.4.2 Controlar la relación bias/varianza



Las grandes redes neuronales son modelos de bajo sesgo. Es importante seleccionar el parámetro de regularización correcto.

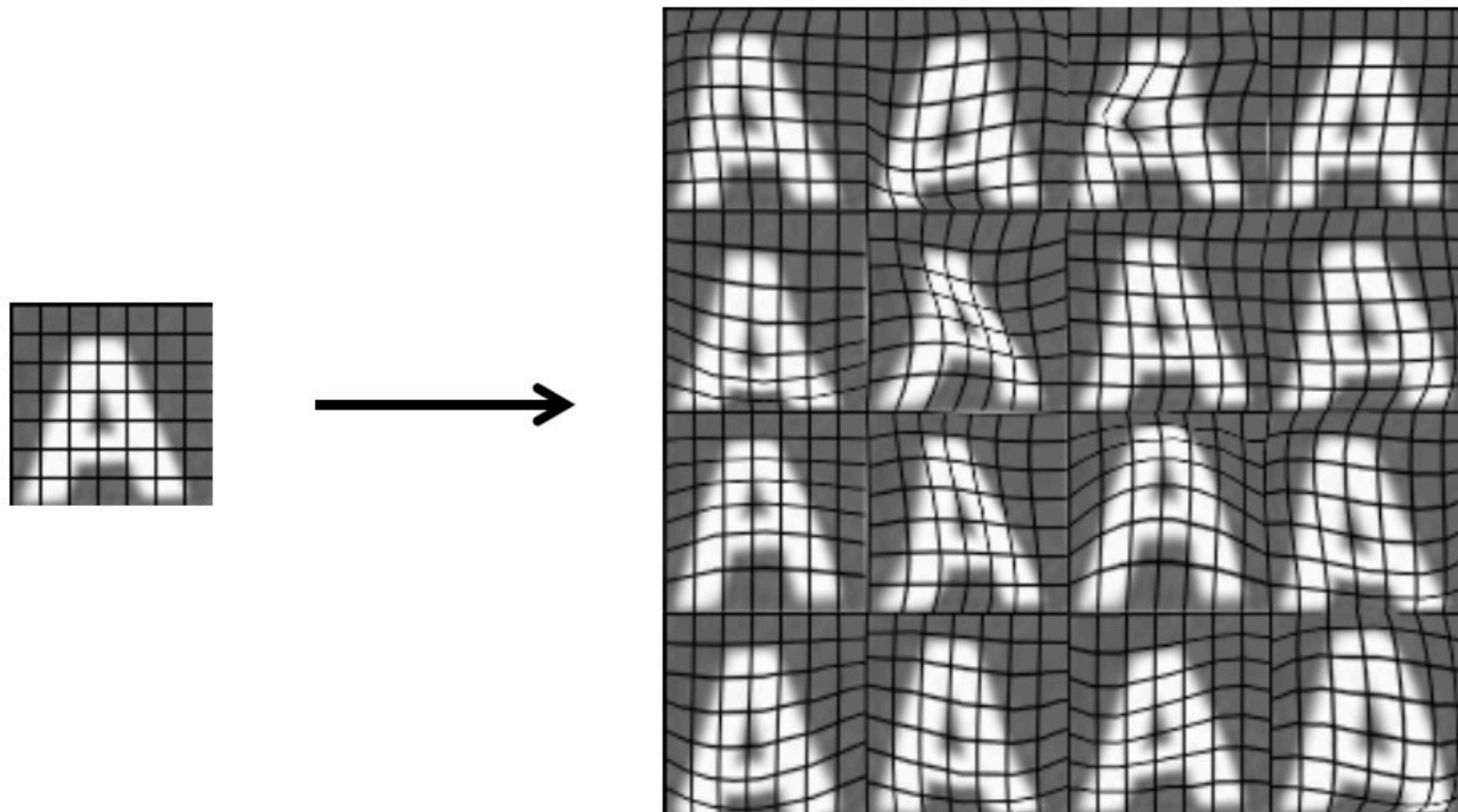
4.5 Añadir más datos

Como ya se ha comentado, añadir más datos de entrenamiento ayuda a reducir la varianza y, por tanto, el error de prueba.

Pero, ¿qué ocurre si no tengo más datos?

Con algunos tipos de datos podemos realizar algunas técnicas que nos ayudan a generar más datos de forma artificial: **Data augmentation**

4.5.1 Data Aumentation usando distorsión



4.5.1 Data Aumentation usando distorsión

Con sonidos, voz, etc., podemos introducir distintos tipos de ruido:

- Ruido de fondo: Maquinaria, Multitud, mala conexión de móvil...

Con imágenes:

- Diferentes tipos de iluminación
- Transformaciones de perspectiva para poder reconocer una imagen transformada

NOTA: No suele ser útil añadir ruido aleatorio o sin sentido a los datos.

Artificial data synthesis for photo OCR

Synthesis: using artificial data inputs to create a new training example.



4.5.2 Ejemplo de data augmentation usando Pytorch:

```
import torch
from torchvision.transforms import v2

H, W = 32, 32
img = torch.randint(0, 256, size=(3, H, W), dtype=torch.uint8)

transforms = v2.Compose([
    v2.RandomResizedCrop(size=(224, 224), antialias=True),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
img = transforms(img)
```

4.5.3 Ejemplo de data augmentation usando keras:

```
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras import layers
# https://www.tensorflow.org/tutorials/images/data\_augmentation?hl=es-419
(train_ds, val_ds, test_ds), metadata = tfds.load('tf_flowers',
split=['train[:80%]', 'train[80%:90%]', 'train[90%:]'], with_info=True, as_supervised=True)
num_classes = metadata.features['label'].num_classes
IMG_SIZE = 180

resize_and_rescale = tf.keras.Sequential([
    layers.Resizing(IMG_SIZE, IMG_SIZE),
    layers.Rescaling(1./255)
])
result = resize_and_rescale(image)
_ = plt.imshow(result)
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
])
```

Asegurarse de que disponemos de un clasificador con un sesgo bajo antes de realizar el esfuerzo de buscar más datos. (Trazad curvas de aprendizaje). Por ejemplo, aumenta el número de características/número de unidades ocultas de la red neuronal hasta que conseguir un clasificador con un sesgo bajo.

Una vez que estemos seguros de que necesitamos más datos, podemos abordar este problema de múltiples maneras:

- Síntesis artificial de datos
- Recopilarlos uno mismo
- Crowdsourcing: Amazon Mechanical Turk <https://www.mturk.com/>

4.6 Transfer Learning

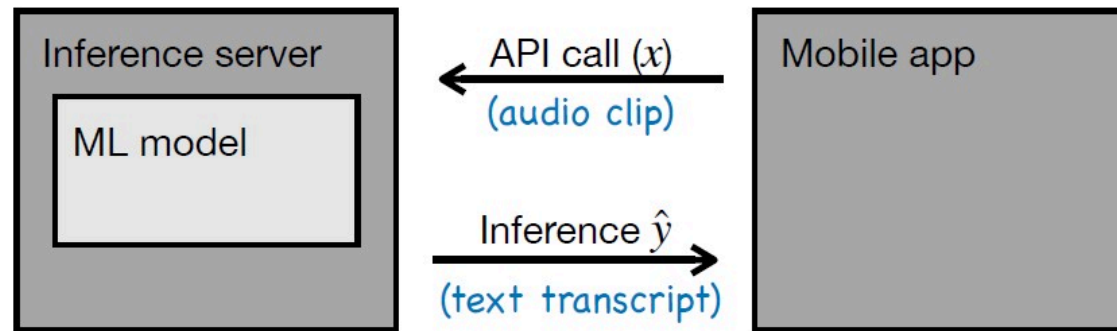
El Transfer Learning: "conjunto de métodos que permiten transferir conocimientos adquiridos gracias a la resolución de problemas para resolver otros problemas."

Las redes de neuronas pueden encadenarse mediante capas. Esta es la base del Deep Learning. Podemos tener un sistema que sea capaz de realizar una tarea, pre-entrenarlo y usar ese sistema ya entrenado con otros datos, añadiendo nuevas capas.

Esas nuevas capas son las que vamos a entrenar, dejando el resto de parámetros con los valores establecidos en el pre-entrenamiento.

4.7 Arquitectura de un sistema de ML

Deployment



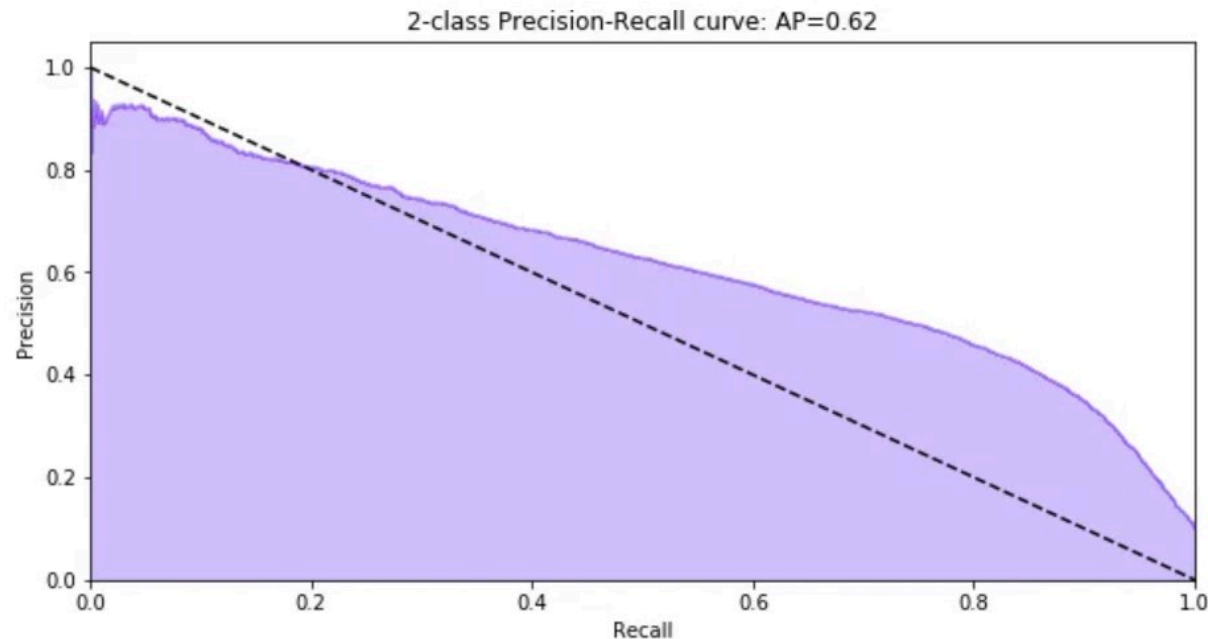
Software engineering may be needed for:

- Ensure reliable and efficient predictions
- Scaling
- Logging
- System monitoring
- Model updates

MLOps
machine learning
operations

4.8 PR curve (curva precision-recall)

Es el resultado de dibujar la gráfica entre el precision y el recall. Nos permite ver a partir de qué recall tenemos una degradación de la precisión y viceversa. Lo ideal sería una curva que se acerque lo máximo posible a la esquina superior derecha (alta precisión y recall)



4.9 Cruva ROC

Una curva ROC (curva de característica operativa del receptor) es otro gráfico que muestra el rendimiento de un modelo de clasificación. Esta curva representa dos parámetros:

- Tasa de verdaderos positivos
- Tasa de falsos positivos

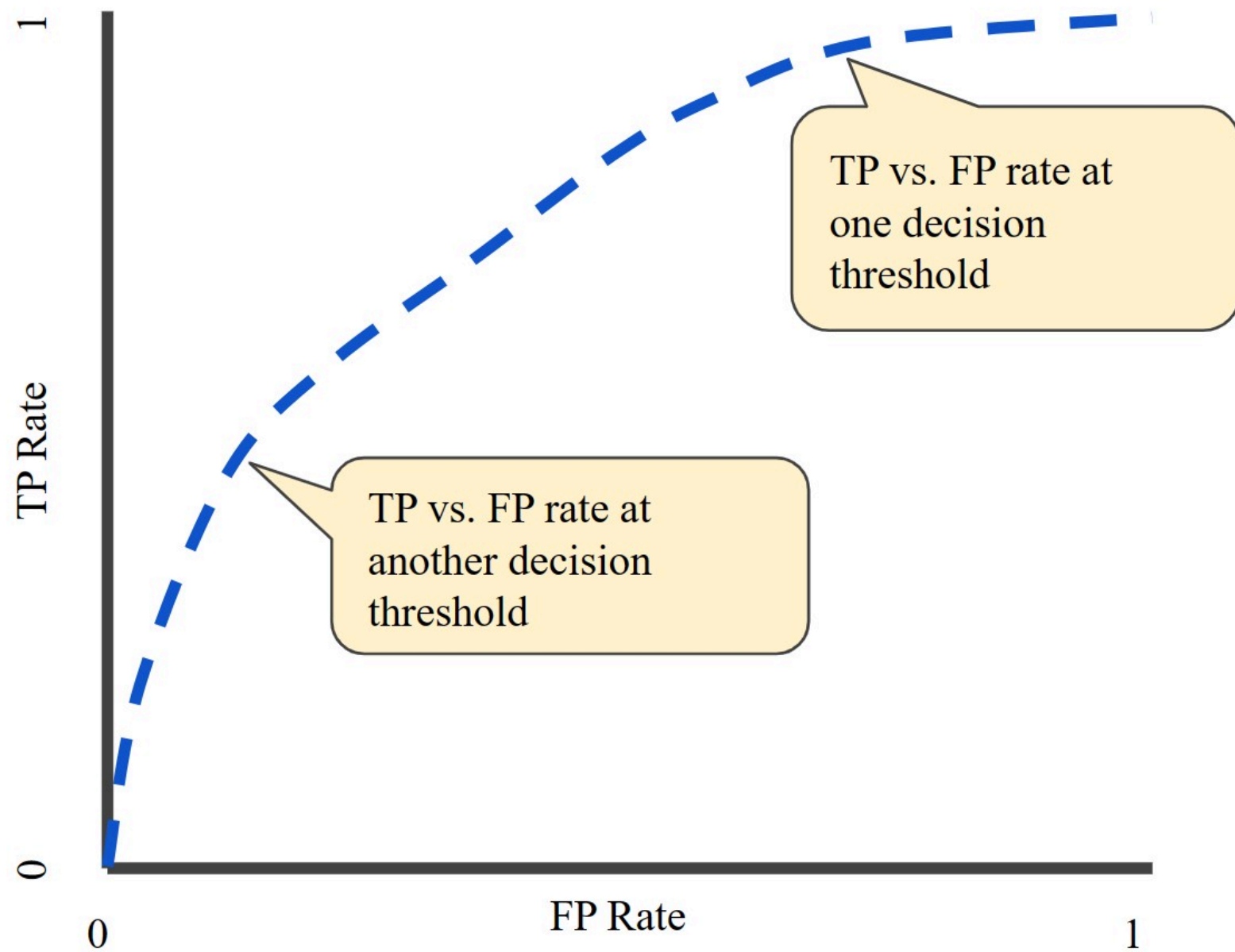
Tasa de falsos positivos (FPR) se define de la siguiente manera:

$$FPR = \frac{FP}{FP + TN}$$

Los clasificadores discretos, como los Árbol de decisión o los sistemas de reglas, dan como resultados a valores numéricos una etiqueta binaria. Estos sólo proporcionan un único punto en el espacio ROC.

Para otros clasificadores, como una Red neuronal artificial o regresión logística, la salida son valores de probabilidad que representan hasta qué punto una instancia pertenece a una de las dos clases.

Para estos métodos se debe fijar un valor **umbral** que determinará un punto en el espacio ROC. Si fijamos el umbral a 0.8, la probabilidad $\geq$ a 0.8 será positiva.



4.9.1 ¿Cómo dibujar la curva ROC?

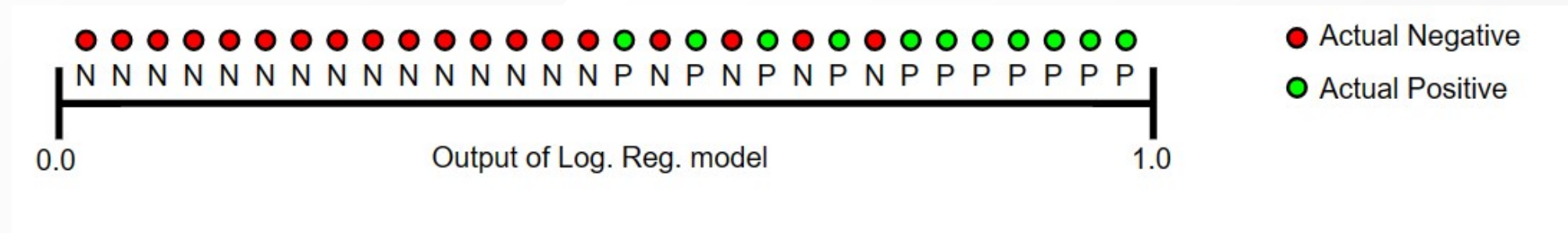
Para cada umbral que decidamos, podemos construir una matriz de confusión, y calcular el punto en el espacio ROC correspondiente.

Según vamos variando el umbral (por ejemplo, en pasos de 0.1) tendríamos una nueva matriz de confusión y un nuevo punto en el espacio ROC. Dibujar la curva ROC consiste en poner juntos todos los puntos correspondientes a todos los umbrales o puntos de corte.

4.9.2 AUC: Área bajo la curva ROC

El AUC proporciona una medida agregada del rendimiento en todos los umbrales de clasificación posibles. Es el indicador más utilizado para determinar la calidad del clasificador.

Se puede interpretar como la probabilidad de que un clasificador ordenará o puntuará una instancia positiva elegida aleatoriamente más alta que una negativa.



4.9.3 Ejemplo de cómo se calcula la curva ROC

```
import numpy as np
from sklearn import metrics
y = np.array([1, 1, 2, 2])
scores = np.array([0.1, 0.4, 0.35, 0.8])
fpr, tpr, thresholds = metrics.roc_curve(y, scores, pos_label=2)
fpr #array([0. , 0. , 0.5, 0.5, 1. ])
tpr #array([0. , 0.5, 0.5, 1. , 1. ])
thresholds #array([ inf, 0.8 , 0.4 , 0.35, 0.1
```

Entrada: salidas reales (0,1) del conjunto de test y las predicciones de probabilidades obtenidas del modelo para la clase 1.

Devuelve: la tasa de falsos positivos y la tasa de verdaderos positivos para cada umbral así como la lista de valores utilizados como umbral.

4.9.4 Ejemplo de cómo se calcula UAC

```
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.metrics import roc_auc_score
>>> X, y = load_breast_cancer(return_X_y=True)
>>> clf = LogisticRegression(solver="liblinear", random_state=0).fit(X, y)
>>> roc_auc_score(y, clf.predict_proba(X)[:, 1]) # 0.99...
# multi_class='ovr' for more than two clases.
roc_auc_score(y, clf.decision_function(X)) # 0.99...
```

Devuelve el valor de AUC ,comprendido entre 0.5 (aleatorio) y 1.0 (perfecto). Modelo con predicciones 100% incorrectas => AUC de 0.0; predicciones 100% correctas => AUC de 1.0.

NOTA: si un clasificador es muy malo (< 0.5) se puede construir un clasificador inverso. No tiene sentido valores por debajo de 0.5

4.9.5 Diferencias entre curvas ROC y PR

Usaremos la curva PR cuando tengamos problemas de datasets no balanceados, es decir, cuando la clase positiva ocurre pocas veces. Cuando hay pocos ejemplos positivos, la curva ROC o el ROC AUC puede dar un valor alto, sin embargo, la curva PR estará lejos de su valor óptimo, poniendo de manifiesto un indicador de precisión relacionado con la baja probabilidad de la clase positiva.

Por el contrario, usaremos la curva ROC y el ROC AUC cuando tengamos un dataset más balanceado o queramos poner de manifiesto un indicador más relacionado con falsas alarmas (falsos positivos).